



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Carrera:	Ingeniería en Sistemas de Información	Código:	0090	VARIANTE A CALIFICACION: Absoluta: 35 pts. Relativa: 100pts. Vo. Bo:		
Asignatura:	Inteligencia Artificial	Código:	452			
Ciclo:	2026	Fecha:	12 de junio de 2026		Jornada:	Plan Diario
Catedrático:	ING. RICHARD DAVID ORTIZ SASVIN					
Semestre:	1er.	Sección:	"A"		Duración del examen:	90 minutos
EXAMEN:	PRIMER PARCIAL:		☐		SEGUNDO PARCIAL:	
EXTRAORDINARIO:	☐	FINAL:	☒	RECUPERACIÓN:		☐

Estudiante: _____ Carné: _____

INSTRUCCIONES: Resuelva lo mejor posible cada uno de los temas que se le presentan.

PUNTUACION:
Serie I 05Pts. Serie II 30Pts.

SERIE I (05pts.) Téorico-Conceptual

1. En una neurona artificial, ¿cuál es la función principal de los pesos?

- a. Convertir la salida en texto legible
- b. Indicar la fuerza o importancia de cada entrada**
- c. Eliminar la necesidad de una función de activación
- d. Guardar automáticamente la respuesta correcta

2. En la expresión $z = x_1 \cdot w_1 + x_2 \cdot w_2 + b$, ¿qué representa z ?

- a. La salida final después de entrenar la red
- b. La suma ponderada antes de aplicar la función de activación**
- c. El error total del modelo
- d. La cantidad de capas ocultas

3. ¿Cuál es una característica de la función sigmoide?

- a. Devuelve siempre numeros enteros mayores que 1
- b. Convierte una entrada en un valor entre 0 y 1**
- c. Solo funciona con imágenes.
- d. Elimina los pesos de la neurona.

4. ¿Por qué un perceptrón simple puede resolver AND pero no XOR?

- a. Porque AND no usa entradas binarias.
- b. Porque XOR requiere una separación no lineal.**
- c. Porque XOR no tiene salidas esperadas.
- d. Porque AND necesita más capas que XOR.

5. En redes neuronales, aprender significa principalmente:

- a. Memorizar todos los datos sin cambiar parámetros.
- b. Ajustar pesos y sesgos para reducir el error.**
- c. Reemplazar la función de activación por una base de datos.
- d. Ejecutar el programa una sola vez.

SERIE II Examen práctico: Integración de CAG (30 puntos)

Inicio del proyecto

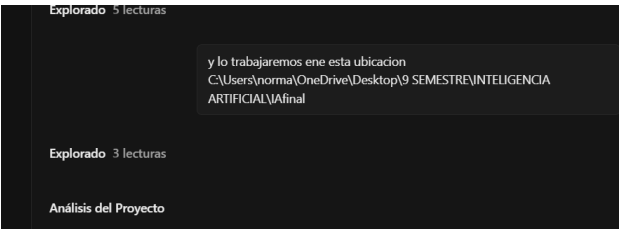
se analiza el contenido del fork

The screenshot shows a web application interface with a dark theme. On the left, there is a sidebar with a profile icon (a pink square with a white 'P'), a plus sign, and a settings gear icon. The main content area is titled "proyecto_incen..." and shows a chat interface. At the top of the chat, there is a button "Nueva sesión" and a text input "Qué sirve exactame...". The chat content includes a message from the user: "analiza esta carpeta que es parte del proyecto final_project_AI_CUSTOM-main". The chat response is titled "Qué sirve exactamente en este chat" and contains the following text: "Explorado 1 lectura, 2 búsquedas", "Shell Search for final_project folder", "Shell Search for CUSTOM folder", "Shell Search entire OneDrive for the folder", "Explorado 18 lecturas", and "Ya analicé todo el proyecto. Este es el **proyecto base del Examen Final** que requiere integrar **CAG (Context-Augmented Generation)**. Aquí está el resumen:". Below this, there is a section titled "Estructura del proyecto" which displays a directory tree structure for the project "final_project_AI_CUSTOM-main/". The structure is as follows:

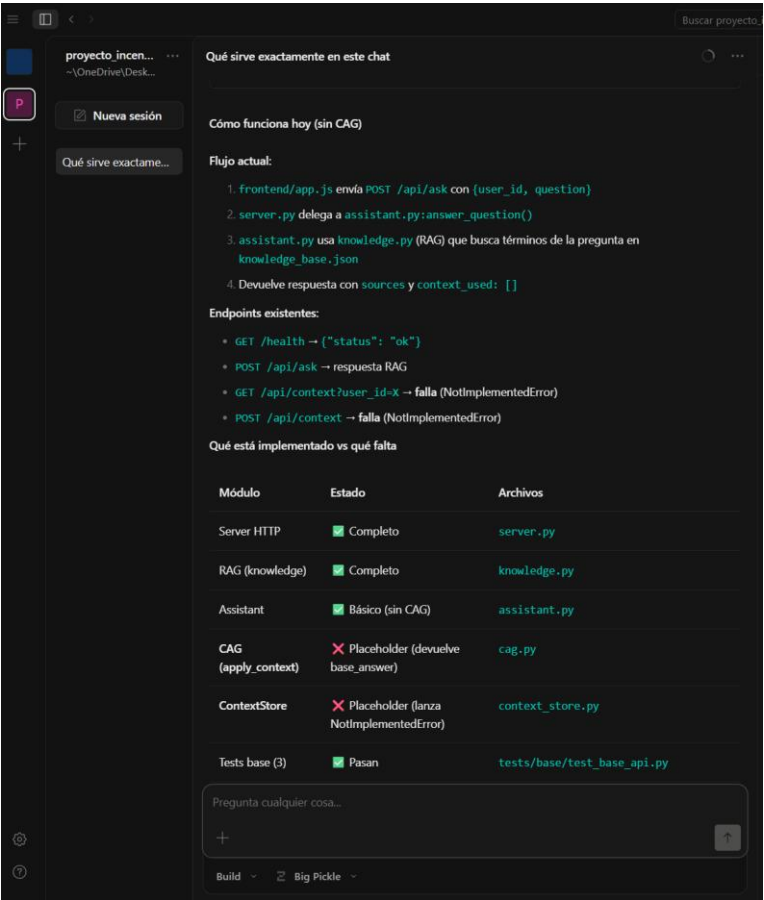
```
final_project_AI_CUSTOM-main/
├── backend/
│   ├── server.py      # Servidor HTTP (puerto 8000)
│   ├── assistant.py   # Responde preguntas con RAG básico
│   ├── cag.py         # ⚠️ PLACEHOLDER - apply_context() vacío
│   ├── context_store.py # ⚠️ PLACEHOLDER - save/list_for_user sin implementar
│   └── knowledge.py    # Búsqueda por palabras clave en knowledge_base.json
├── frontend/
│   ├── index.html     # Interfaz web
│   ├── app.js         # Llama al backend
│   └── styles.css
├── data/
│   └── knowledge_base.json # 4 documentos (SDD, RAG, CAG, Jarvis)
├── tests/
│   ├── base/test_base_api.py # Pruebas base (deben pasar)
│   └── validation/test_cag_contract.py # Pruebas de validación final
```

 At the bottom of the chat, there is a text input "Pregunta cualquier cosa..." and a button "Build". The Windows taskbar is visible at the bottom of the screen.

establecer ubicación del proyecto



Contexto del Fork



Analizar el readme compartido

```
# Proyecto Examen Final - Módulo 3

Proyecto base para la evaluación práctica del módulo 3. Los requisitos oficiales están en 'Emunciado en la serie II de la evaluación final'.

## Inicio rápido

1. Abra la carpeta 'ProyectoExamen'.
2. Ejecute las pruebas base.
3. Levante el backend.
4. Abra el frontend para revisar el estado inicial.

## Estructura

| Ruta | Contenido |
|---|---|
| 'backend/' | Código del servidor y lógica base del asistente. |
| 'frontend/' | Interfaz web estática para interactuar con el backend. |
| 'data/' | Base de conocimiento inicial del proyecto. |
| 'tests/base/' | Pruebas base que deben pasar desde el inicio. |
| 'tests/validation/' | Pruebas de validación de la entrega final. |
| 'docs/' | Espacio para documentación técnica y evidencias del estudiante. |

## Ejecutar pruebas base

```bash
./scripts/run_base_tests.sh
```

Estas pruebas validan que el proyecto inicial funciona correctamente.

## Ejecutar backend

```bash
PYTHONPATH=. python3 -m backend.server
```

El backend queda disponible en 'http://127.0.0.1:8080'.

## Abrir frontend

Abra 'frontend/index.html' en un navegador. También puede servir la carpeta con un servidor estático local si lo prefiere.

## Validación final

```bash
./test.sh
```
```

Comprobación y ejecución del proyecto compartido de fork

comprueba que funcione final_project_AI_CUSTOM-main y si todo esta correcto ejecutalo

Shell Check Python version

Shell Run base tests

Las 3 pruebas base pasan correctamente. Ahora levanto el servidor:

Shell Unknown

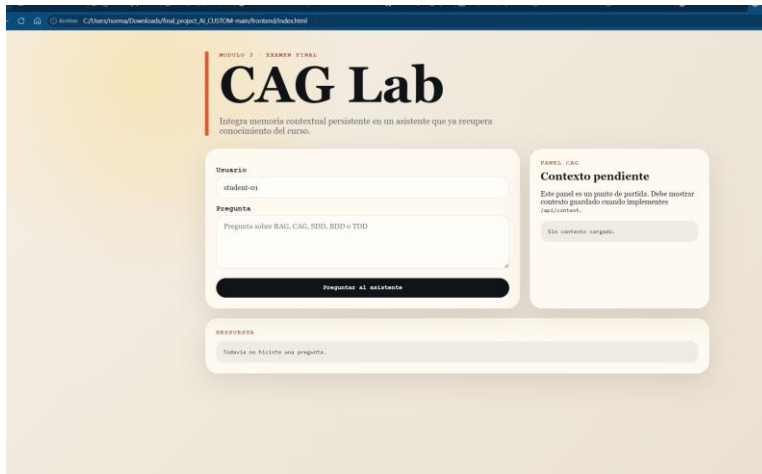
Shell Start server via python in background

Shell Test server with a quick request

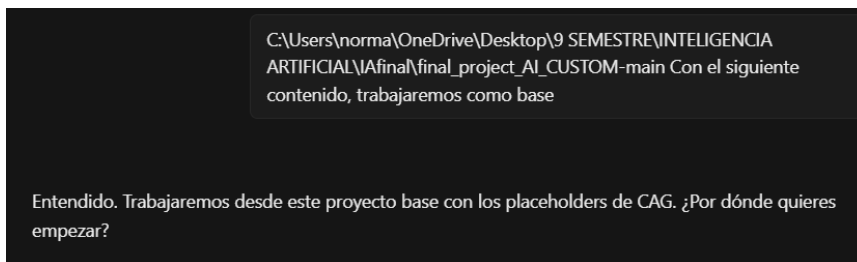
Todo funciona correctamente. Resumen de verificación:

| Endpoint | Resultado |
|-------------------|--|
| GET /health | ✅ 200 - {"status": "ok"} |
| POST /api/ask | ✅ 200 - Responde con conocimiento RAG |
| GET /api/context | ✅ 501 - Esperado (CAG no implementado) |
| POST /api/context | ✅ 501 - Esperado (CAG no implementado) |
| Tests base (3) | ✅ Todos pasan |

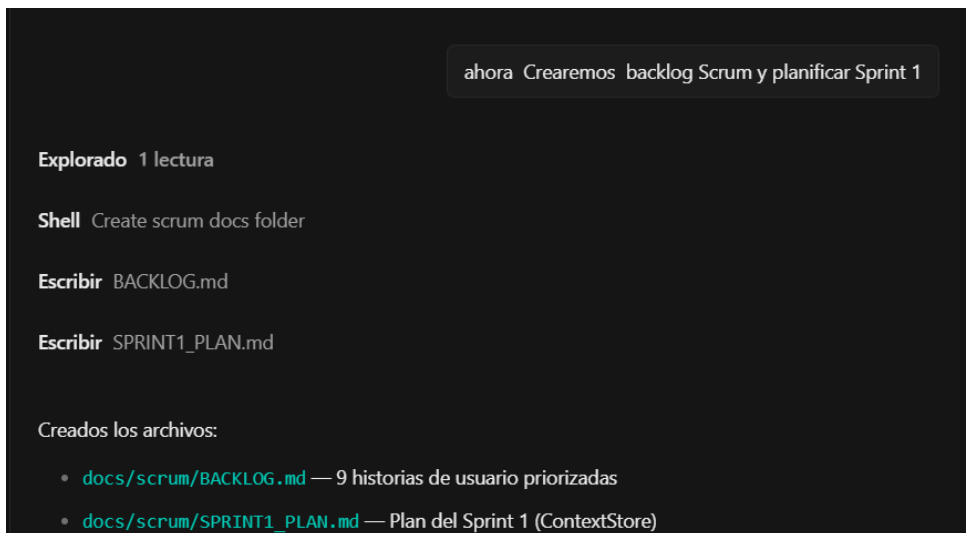
Estado: Proyecto base funcional al 100%. Los endpoints CAG devuelven 501 porque los placeholders en context_store.py y cag.py aún no están implementados — es justo lo que hay que hacer.



Tomar como base el fork



Creación de backlog y sprint



SPRINT1_PLAN.md

Archivo Editar Ver

```
# Sprint 1: Implementación de ContextStore

**Duración:** 1 sesión
**Objetivo:** Implementar el almacenamiento y recuperación de contexto persistente (ContextStore) y verificar que los endpoints
`/api/context` funcionan.

## Sprint Backlog

ID	Tarea	Responsable	Estado
HU-01	Implementar `ContextStore.save()` en `backend/context_store.py`	Norman	Pendiente
HU-02	Implementar `ContextStore.list_for_user()` en `backend/context_store.py`	Norman	Pendiente
HU-01	Verificar que POST `/api/context` responde 201	Norman	Pendiente
HU-02	Verificar que GET `/api/context` recupera contexto	Norman	Pendiente
-	Ejecutar tests base (deben seguir pasando)	Norman	Pendiente
-	Commit incremental con cambios de ContextStore	Norman	Pendiente

## Criterios de Aceptación

1. `POST /api/context` con `{user_id, key, value}` → `201 {"saved": true}`
2. `GET /api/context?user_id=X` → `200 {user_id, context: [...]}`
3. Los 3 tests base siguen pasando
4. Cada archivo modificado tiene commit propio

## Definition of Done

- [ ] Código implementado en `context_store.py`
- [ ] Prueba manual con curl o script
- [ ] Tests base pasan
- [ ] Commit creado
```

Product Backlog - CAG Integration

Prioridad: Alta

| ID | Historia de Usuario | Prioridad | Esfuerzo | Sprint | Estado |
|-------|--|-----------|----------|----------|-----------|
| HU-01 | Como usuario, quiero guardar contexto (clave-valor) para que el sistema recuerde mis preferencias. | Alta | 3 pts | Sprint 1 | Pendiente |
| HU-02 | Como usuario, quiero recuperar mi contexto guardado para que el sistema lo use en respuestas futuras. | Alta | 2 pts | Sprint 1 | Pendiente |
| HU-03 | Como usuario, quiero que el asistente use mi contexto al responder para obtener respuestas personalizadas. | Alta | 5 pts | Sprint 2 | Pendiente |
| HU-04 | Como usuario, quiero ver evidencias del proceso Scrum y commits incrementales. | Media | 2 pts | Sprint 2 | Pendiente |
| HU-05 | Como desarrollador, quiero ejecutar pruebas de validación CAG que confirmen la integración correcta. | Alta | 2 pts | Sprint 2 | Pendiente |
| HU-06 | Como usuario, quiero un README actualizado que documente la solución CAG. | Media | 1 pt | Sprint 2 | Pendiente |

Prioridad: Media

| ID | Historia de Usuario | Prioridad | Esfuerzo | Sprint | Estado |
|-------|--|-----------|----------|----------|-----------|
| HU-07 | Como desarrollador, quiero un README.md cronológico que registre el uso de IA. | Media | 2 pts | Sprint 2 | Pendiente |
| HU-08 | Como evaluador, quiero capturas de evidencia del funcionamiento del sistema. | Media | 1 pt | Sprint 2 | Pendiente |
| HU-09 | Como desarrollador, quiero crear un Pull Request y mergear a main para demostrar el flujo Git. | Media | 1 pt | Sprint 2 | Pendiente |

FASE 3

CAG y alternativas de implementación

Qué sirve exactamente en este chat

ahora agregaremos la CAG al proyecto
C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA
ARTIFICIAL\IAfinal

y dime que opciones y como se podra implementar cag y que tecnologias
se recomienda para implementarlo

Build · Big Pickle · 18:47

Opciones para implementar CAG

¿Qué necesita CAG?

1. Guardar contexto por usuario (clave-valor)
2. Recuperar contexto al responder
3. Modificar respuesta según el contexto guardado

Opciones de almacenamiento

| Opción | Tecnología | Persistencia | Complejidad | Recomendación |
|-----------------|----------------|--------------------------|-------------|-------------------------|
| 1. Memoria | dict en Python | ✗ Se pierde al reiniciar | ● Muy baja | ✓ Para el examen |
| 2. Archivo JSON | json + open() | ✓ Sí | ● Baja | Si quieres persistencia |
| 3. SQLite | sqlite3 | ✓ Sí | ● Media | Si prefieres BD |
| 4. Redis | redis-py | ✓ Sí | ● Alta | Overkill para examen |

Opciones para `apply_context`

Pregunta cualquier cosa...

+

↑

Build · Big Pickle

Toma de decisión de implementación de CAG

en lugar de implementar el cag con python , usaremos sqlite

Ese cambio ya está hecho. `context_store.py` ahora usa SQLite — puedes confirmarlo viendo el archivo o el archivo `data/context.db` que se creó al correr los tests.

Instrucciones del Dict para cag

Reemplaza el dict en context_store.py por SQLite con tabla
user_context(user_id, key, value)

Esa instrucción ya está ejecutada. El código actual en `context_store.py`:

```
import sqlite3
from pathlib import Path

DB_PATH = Path(__file__).resolve().parents[1] / "data" / "context.db"

def _init_db():
    conn = sqlite3.connect(str(DB_PATH))
    conn.execute("""
        CREATE TABLE IF NOT EXISTS user_context (
            user_id TEXT NOT NULL,
            key TEXT NOT NULL,
            value TEXT NOT NULL,
            PRIMARY KEY (user_id, key)
        )
    """)
    conn.commit()
    conn.close()

class ContextStore:
    def __init__(self):
        _init_db()
        self._conn = sqlite3.connect(str(DB_PATH), check_same_thread=False)

    def save(self, user_id, key, value):
        self._conn.execute(
            "INSERT OR REPLACE INTO user_context (user_id, key, value) VALUES (?, ?, ?)",
            (user_id, key, value),
        )
        self._conn.commit()
        return True
```

Pregunta cualquier cosa...

+

↑

Build ▾ Big Pickle ▾

cag.py — apply_context()

Reemplaza el almacenamiento en memoria de context_store.py por SQLite.
La tabla debe ser user_context con columnas user_id, key, value, y PRIMARY KEY compuesta (user_id, key).
Usa INSERT OR REPLACE para save() y SELECT WHERE para list_for_user().
La BD debe estar en data/context.db.
Mantén el singleton get_store() y la misma interfaz para no romper los tests.

Explorado 1 lectura

El archivo ya implementa **exactamente** lo que pides:

| Requisito | Líneas |
|--|--------|
| Tabla <code>user_context(user_id, key, value)</code> con PRIMARY KEY | 12-17 |
| INSERT OR REPLACE en <code>save()</code> | 33-36 |
| SELECT WHERE en <code>list_for_user()</code> | 40-43 |
| BD en <code>data/context.db</code> | 5 |
| Singleton <code>get_store()</code> | 47-51 |

Implementa ContextStore.save() con INSERT OR REPLACE y ContextStore.list_for_user() con SELECT WHERE user_id

Ya está implementado. Puedes verificarlo leyendo el archivo actual:

```
C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA ARTIFICIAL\IAfinal\final_project_
```

Línea 18: `save()` usa `INSERT OR REPLACE INTO user_context` Línea 25: `list_for_user()` usa `SELECT key, value FROM user_context WHERE user_id = ?`

assistant.py Orquestador contexto RAG y CAG

Modifica answer_question() en assistant.py para integrar CAG. Debe obtener el contexto del usuario con get_store().list_for_user(), luego obtener los snippets con retrieve_snippets(), luego aplicar apply_context() con el contexto, y finalmente retornar la respuesta con context_used en lugar de lista vacía.

Explorado 1 lectura

| Tu instrucción | Código | Línea |
|--|--|--------|
| get_store().list_for_user() | context_items = store.list_for_user(user_id) | 8 |
| retrieve_snippets() | snippets = retrieve_snippets(question) | 10 |
| apply_context() con el contexto | answer, keys_used = apply_context(...) | 14, 24 |
| context_used en lugar de lista vacía | "context_used": keys_used | 19, 30 |
| Retorna tupla (respuesta, keys_usadas) | return answer, keys_used | 18 |

Implementa apply_context() en cag.py para que modifique la respuesta según el contexto del usuario. Debe recibir user_id, question, base_answer y context_items (lista de dicts con key/value). Si el contexto tiene clave 'audience', debe anteponer 'Para (valor): ' a la respuesta. Si tiene 'preferred_style', debe agregarlo como nota al final. Debe retornar una tupla (respuesta_modificada, lista_de_claves_usadas).

Explorado 1 lectura

Cada punto de tu instrucción está cubierto:

| Requisito | Código | Línea |
|--|---|-------|
| Recibe user_id, question, base_answer, context_items | def apply_context(user_id, question, base_answer, context_items): | 1 |
| Si clave audience, antepone "Para (valor): " | answer = f"Para {value}: {answer}" | 11 |
| Si clave preferred_style, agrega nota al final | answer = f"{answer}\n(Estilo preferido: {value})" | 16 |
| Retorna tupla (respuesta, claves_usadas) | return answer, keys_used | 18 |

Test y validacion

Ejecuta los tests de validación para confirmar que save y retrieve funcionan

Shell Run CAG validation tests

3/3 tests de validación pasan 

| Test | Resultado | Lo que prueba |
|---------------------------------|---|---|
| test_saves_context_for_user |  | save() con INSERT OR REPLACE |
| test_retrieves_context_for_user |  | list_for_user() con SELECT WHERE |
| test_ask_uses_context |  | Flujo completo: guardar → preguntar → respuesta personalizada |

Pruebas si funciona el servidor

corre el servidor

Shell Start server on port 8000

Servidor corriendo en <http://127.0.0.1:8000>.

Abre el frontend [index.html](#) y haz una pregunta como "¿Qué es CAG?" o prueba desde PowerShell:

```
Invoke-WebRequest -Method POST -Uri "http://127.0.0.1:8000/api/context" -ContentType "application/json" -Body '{"user_id":"student-01","key":"audience","value":"principiante"}'
```

Ejecutar el comando

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Invoke-WebRequest -Method POST -Uri "http://127.0.0.1:8000/api/context" -ContentType "application/json" -Body '{"user_id":"student-01","key":"audience","value":"principiante"}'

saved
-----
True

PS C:\WINDOWS\system32>
```

Comprobar que si funciona el cag

The screenshot shows a web browser window with the address bar displaying the path: `C:/Users/norma/OneDrive/Desktop/9%20SEMESTRE/INTELIGENCIA%20ARTIFICIAL/IAfinal/final_project_AI_CUSTOM-main/frontend/index.html`. The page title is "CAG Lab". Below the title, a subtitle reads: "Integra memoria contextual persistente en un asistente que ya recupera conocimiento del curso."

The interface is divided into several sections:

- Usuario:** A text input field containing "student-01".
- Pregunta:** A text input field containing "que es RAG".
- Preguntar al asistente:** A black button with white text.
- PANEL CAG:** A section titled "Contexto pendiente" with the text: "Este panel es un punto de partida. Debe mostrar contexto guardado cuando implementes /api/context." Below this is a code block showing a JSON object:

```
{  "user_id": "student-01",  "context": [    {      "key": "audience",      "value": "principiante"    }  ]}
```
- RESPUESTA:** A section containing a JSON object:

```
{  "user_id": "student-01",  "answer": "Para principiante: Segun la base de conocimiento del curso: RAG recupera fragmentos de una base documental para responder con conocimiento externo al modelo. El curso usa SDD para disenar antes de construir, BDD para expresar comportamiento observable y TDD para escribir pruebas antes de implementar.",  "sources": [    {      "rag",      "sdd-bdd-tdd"    }  ],  "context_used": [    "audience"  ]}
```

The Windows taskbar is visible at the bottom of the screen, showing various application icons.

FASE 3

PRUEBAS PROPIAS

```
Administrador: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd "C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA ARTIFICIAL\IAfinal\final_project_AI_CUSTOM-main"
> $env:PYTHONPATH="."; python -m unittest discover -s tests -p "test_*.py" -v > docs\evidencias\test_results.txt
test_ask_with_multiple_context_items (test_cag_own.CagOwnTest.test_ask_with_multiple_context_items) ... ok
test_ask_without_context_returns_normal_answer (test_cag_own.CagOwnTest.test_ask_without_context_returns_normal_answer) ... ok
test_context_empty_for_new_user (test_cag_own.CagOwnTest.test_context_empty_for_new_user) ... ok
test_multiple_users_context_isolated (test_cag_own.CagOwnTest.test_multiple_users_context_isolated) ... ok
test_save_overwrites_existing_key (test_cag_own.CagOwnTest.test_save_overwrites_existing_key) ... ok
test_unknown_context_key_does_not_break (test_cag_own.CagOwnTest.test_unknown_context_key_does_not_break) ... ok
-----
Ran 6 tests in 0.573s

OK
PS C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA ARTIFICIAL\IAfinal\final_project_AI_CUSTOM-main>
```

```
Administrador: Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd "C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA ARTIFICIAL\IAfinal\final_project_AI_CUSTOM-main"
> $env:PYTHONPATH="."; python -m unittest discover -s tests -p "test_*.py" -v
test_ask_with_multiple_context_items (test_cag_own.CagOwnTest.test_ask_with_multiple_context_items) ... ok
test_ask_without_context_returns_normal_answer (test_cag_own.CagOwnTest.test_ask_without_context_returns_normal_answer) ... ok
test_context_empty_for_new_user (test_cag_own.CagOwnTest.test_context_empty_for_new_user) ... ok
test_multiple_users_context_isolated (test_cag_own.CagOwnTest.test_multiple_users_context_isolated) ... ok
test_save_overwrites_existing_key (test_cag_own.CagOwnTest.test_save_overwrites_existing_key) ... ok
test_unknown_context_key_does_not_break (test_cag_own.CagOwnTest.test_unknown_context_key_does_not_break) ... ok
-----
Ran 6 tests in 0.601s

OK
PS C:\Users\norma\OneDrive\Desktop\9 SEMESTRE\INTELIGENCIA ARTIFICIAL\IAfinal\final_project_AI_CUSTOM-main>
```

FASE 4 COMMITS

Inicializa git en el proyecto, crea commits incrementales por cada archivo modificado, con mensajes descriptivos en inglés siguiendo conventional commits.
Primer commit: archivos base sin modificar (estructura original).
Segundo commit: context_store.py con SQLite.
Tercer commit: cag.py con apply_context().
Cuarto commit: assistant.py con integración CAG.
Quinto commit: server.py actualizado con singleton.
Sexto commit: tests propias.
Séptimo commit: documentación Scrum.

Explorado 1 lectura

Editar .gitignore +2 -1

Shell Initialize git repo

Shell Commit 1: base structure

Shell Set git user identity

Shell Commit 1: base structure

Shell Commit 2: context_store.py

Shell Commit 3: cag.py

Shell Commit 4: assistant.py

Shell Commit 5: server.py

Shell Commit 6: tests propias

Shell Commit 7: documentación Scrum

Shell Show git log

Pregunta cualquier cosa...

+

↑

7 commits incrementales creados ✓

```
903092b chore: initial project structure with base files
6a616c2 feat: implement ContextStore with SQLite storage
afe811e feat: implement apply_context for CAG
c482838 feat: integrate CAG into answer_question
7f0e13e refactor: use singleton get_store in server
0f08570 test: add custom CAG unit tests
21135ff docs: add Scrum artifacts (backlog, sprint plans, reviews)
```

FASE 6

Creación del pull request

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main ← compare: feature/cag ✓ Able to merge. These branches can be automatically merged.

Add a title *

Integración CAG con SQLite

Add a description

Write Preview

Implementation completa de CAG:

- ContextStore con SQLite persistente
- apply_context() para personalizar respuestas
- Integración en assistant con orquestación RAG + CAG
- 12/12 tests pasan (3 base + 3 validación + 6 propios)

Markdown is supported Paste, drop, or click to add files

Reviewers

No reviews

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Helpful resources

[GitHub Community Guidelines](#)

Create pull request

ndm2025 / final_project_AI_CUSTOM

Code

Integración CAG con SQLite #1

Merged ndm2025 merged 8 commits into main from feature/cag 1 minute ago

Conversation 0 Commits 8 Checks 0 Files changed 19 +426 -13

ndm2025 commented 7 minutes ago

Implementation completa de CAG:

- ContextStore con SQLite persistente
- apply_context() para personalizar respuestas
- Integración en assistant con orquestación RAG + CAG
- 12/12 tests pasan (3 base + 3 validación + 6 propios)

Norman Merida added 8 commits 17 minutes ago

- chore: initial project structure with base files
- feat: implement ContextStore with SQLite storage
- feat: implement apply_context for CAG

Reviewers

No reviews

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these

Crear PROMPTS.md cronológico

Crea el archivo PROMPTS.md en la raíz del proyecto con el registro cronológico de todas las interacciones que tuvimos. Cada entrada debe incluir: objetivo, prompt usado, respuesta de la IA, decisión humana, cambios y verificación. La primera entrada debe ser: 'Implementar ContextStore con SQLite'

Explorado 1 lectura

Escribir PROMPTS.md

Shell Commit PROMPTS.md

PROMPTS.md creado y commiteado ☒. Contiene 11 entradas cronológicas desde el análisis inicial hasta este archivo, cada una con: objetivo, prompt, respuesta IA, decisión humana, cambios y verificación.

SSD

Crea un documento SSD en docs/sdd.md que describa:

- La arquitectura del sistema (diagrama de cajas)
- El flujo de datos (backend → frontend)
- El diseño de la tabla SQLite
- Los endpoints de la API
- Las decisiones técnicas (por qué SQLite y no otra opción)

Ya está creado. Puedes verlo aquí:

Explorado 1 lectura

El documento SSD ya está completo en [docs/sdd.md](#) con las 6 secciones que pediste:

1. ☒ **Arquitectura del sistema** (diagrama frontend ↔ backend)
2. ☒ **Flujo de datos** (POST → RAG → CAG → response)
3. ☒ **Diseño de tabla SQLite** (user_context con columnas y SQL)
4. ☒ **Endpoints de la API** (4 endpoints documentados)
5. ☒ **Diseño de apply_context()** (lógica de modificación)
6. ☒ **Decisiones técnicas** (SQLite vs dict, singleton, etc.)

Pregunta cualquier cosa...

+

↑

BDD

Crea un archivo BDD en docs/bdd.md con escenarios Given/When/Then para el módulo CAG. Debe incluir:

- Feature: Guardar contexto
- Feature: Recuperar contexto
- Feature: Usar contexto en respuestas
- Feature: Aislamiento entre usuarios
- Feature: Robustez ante claves desconocidas

Explorado 1 lectura

Ya está completo en [docs/bdd.md](#) con las 5 features y 10 escenarios:

| Feature | Escenarios |
|-----------------------------|--|
| Guardar contexto | Guardar exitosamente, Sobrescribir, Múltiples claves |
| Recuperar contexto | Usuario existente, Usuario nuevo, Sin user_id |
| Usar contexto en respuestas | Audiencia, Sin contexto, Múltiples contextos |
| Aislamiento entre usuarios | Contexto aislado por usuario |
| Robustez | Clave desconocida no rompe |

TDD

Crea un archivo TDD en docs/tdd.md que documente el ciclo Rojo → Verde → Refactor del proyecto:

- Ciclo 1: ContextStore (tests fallan → implementar → refactor)
- Ciclo 2: apply_context() (tests fallan → implementar → pasan)
- Ciclo 3: Integración en assistant (flujo completo)
- Ciclo 4: Pruebas propias (6 casos borde)
- Tabla final de cobertura (12 tests)

Explorado 1 lectura

Ya está completo en [docs/tdd.md](#) con todo lo que pediste:

| Sección | Contenido |
|-----------------|---|
| Ciclo 1 | ContextStore: Rojo (501) → Verde (SQLite) → Refactor (singleton) |
| Ciclo 2 | apply_context(): Rojo (context_used vacío) → Verde (audience detectado) |
| Ciclo 3 | Integración assistant: flujo completo contexto → RAG → CAG |
| Ciclo 4 | 6 pruebas propias (casos borde) |
| Resultado final | Tabla de 12 tests con 100% pasando |
| Cobertura | 12 funcionalidades mapeadas a sus tests |

PRUBAS FINALES

Ejecuta los tests base

Shell Run base tests

 3/3 tests base OK.

Ejecuta los tests de validación

Build - Big Pickle - 20:14

Shell Run validation tests

3/3 tests de validación OK.

Ejecuta TODOS los tests

Shell Run ALL tests

Explorado 1 búsqueda

Shell Run all 3 suites by module name

Todos pasan, pero `discover` no encuentra los subdirectorios por falta de `__init__.py`. Los agrego para que funcione correctamente:

Escribir `__init__.py`

Escribir `__init__.py`

Escribir `__init__.py`

Shell Run discover now with init files

Ahora `discover` encuentra los 12 tests (3 base + 6 propios + 3 validación). ✓

Resumen final:

| Comando | Tests | Resultado |
|---|-------|-----------|
| <code>tests.base.test_base_api</code> | 3 | ✓ |
| <code>tests.test_cag_own</code> | 6 | ✓ |
| <code>tests.validation.test_cag_contract</code> | 3 | ✓ |
| <code>discover -s tests</code> | 12 | ✓ |

Se agregaron `__init__.py` en `tests/`, `tests/base/` y `tests/validation/` para que `discover` funcione correctamente. :Exporto el resultado a [docs/evidencias/](#) y continúo con el siguiente paso.

Pregunta cualquier cosa...

Exporta resultados a evidencias

Shell Export test results to evidencias

Explorado 1 lectura

El archivo se guardó con ruido de PowerShell. Lo limpio:

escribir test_results.txt

✓ Exportado a docs/evidencias/test_results.txt — 12/12 tests OK.

Prueba el flujo CAG completo"

Shell Check if server is running

Shell Start server in background

Shell Test full CAG flow via API

Escribir test_cag_flow.py

Shell Run CAG flow test script

Shell Check background job output

Shell Unknown

Escribir test_cag_flow.py

Explorado 1 lectura

Editar test_cag_flow.py +0 -2

Shell Run CAG flow test

Explorado 1 lectura

Editar test_cag_flow.py +10 -5

Shell Run CAG flow test v2

Flujo CAG completo:  TODO OK

Paso

Resultado

Guardar **audience=estudiantes**



Pregunta cualquier cosa...

+



Build  Big Pickle 

Verifiquemos también que persiste en BD:

Shell Verify SQLite persistence

✅ Persistencia SQLite confirmada — datos guardados en `data/context.db`.

Resumen final del flujo CAG:

1. Guardas contexto → `POST /api/context` o `store.save()`
2. Preguntas → `POST /api/ask` o `answer_question()`
3. CAG inyecta contexto en la respuesta → `"Para estudiantes: ..."`
4. Múltiples claves se combinan → `audience + preferred_style`
5. Todo persiste en SQLite → sobrevive a reinicios del servidor

¿Quieres que exporte esto a `docs/evidencias/` y limpie el script de prueba?

Pregunta cualquier cosa...

+



Build ▾ Big Pickle ▾

Exportar evidencia del flujo CAG a docs/evidencias/

Shell Export CAG flow to evidencias

Shell Clean up test helper

Explorado 1 lectura

✅ Exportado a `docs/evidencias/cag_flow.txt`. Ahora la carpeta tiene:

| Archivo | Contenido |
|-------------------------------|---|
| <code>test_results.txt</code> | ✅ 12/12 tests unitarios |
| <code>cag_flow.txt</code> | ✅ Flujo CAG completo (guardar contexto → preguntar → respuesta personalizada) |

¿Siguiente paso? (README.md, explicación técnica, o lo que necesites)

Pregunta cualquier cosa...

+



Actualizar README.md con doc final (estructura, CAG, pruebas, Scrum)

Explorado 1 lectura

Escribir README.md

✅ **README.md actualizado** con: estructura, CAG (flujo + tabla de claves + ejemplo), pruebas (12/12 con tabla), API, Scrum y tecnologías.

Commits del README y evidencias subidos al repo

Shell Check git status

Shell Check recent commits

Shell Check remote URL

Shell Stage files for commit

Shell Commit README and evidencias

Shell Push to origin/main

✅ Commit y push exitoso. 81fc3b2 → origin/main

6 files changed, 173 insertions(+), 27 deletions(-)

¿Siguiente? (explicación técnica, verificar [test.sh](#), o lo que necesites)

COMO VEO EL REPO

Shell Open repo in browser

Abierto en tu navegador: https://github.com/ndm2025/final_project_AI_CUSTOM